

BINARIZZAZIONE DI UNA MATRICE

Gregorio Bussolari

OBIETTIVO

Si vuole trasformare una matrice quadrata $A[N] \times [N]$ di $N \times N$ valori reali, in una matrice di interi binaria $T[N][N]$.

Pertanto:

per ogni $a_{ij} \in A, i \in 0, N - 1, j \in 0, N - 1$ si consideri il suo intorno $A_{ij}[3][3]$

Ogni $t_{ij} \in T, i \in 0, N - 1, j \in 0, N - 1$ avrà un valore calcolato con il seguente criterio:

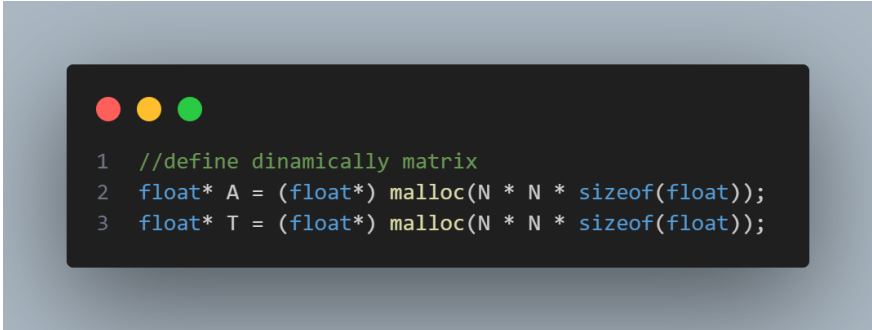
- Se $a_{ij} > m_{ij}$: $t_{ij} = 1$
 - se $a_{ij} \leq m_{ij}$: $t_{ij} = 0$

IMPLEMENTAZIONE MPI

- Il nodo “master” con rank = 0, assegna ad ogni nodo la porzione di matrice A su cui lavorare
- Ogni nodo calcola una diversa porzione della matrice risultato T
- Al termine il nodo master aggatherà i risultati prodotti da tutti i nodi nella matrice risultato T

INIZIALIZZAZIONE

Matrici allocate usando l’approccio Row Major, inizializzate a valori randomici, con $N \geq 2000$



```
1 //define dinamicamente matrix
2 float* A = (float*) malloc(N * N * sizeof(float));
3 float* T = (float*) malloc(N * N * sizeof(float));
```

DIVISIONE DEL LAVORO

- Il calcolo della matrice T è stato diviso tra i vari nodi al fine di garantire un workload bilanciato, calcolando num_rows elementi di T
- Ogni processo riceve la porzione della matrice A attraverso la primitiva di comunicazione collettiva, MPI_Scatterv: scatter con blocchi di dimensione variabile, assegnandola all'array locale 'my_A'

Rispetto alla MPI_Scatter necessita di 2 array, definiti dal sender:

- **-sendcounts:** integer array (of length group size) specifying the number of elements to send to each processor
- **Displs:** integer array (of length group size). Entry i specifies the displacement (relative to sendbuf from which to take the outgoing data to process).



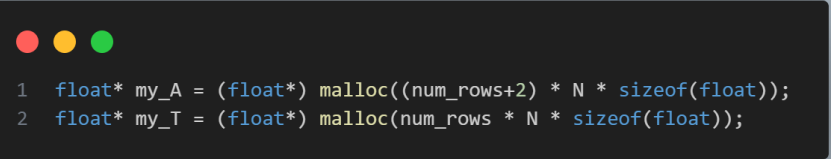
```
1 int num_rows = N / size;
2 int resto = N % size;
3 if(rank < resto){
4     num_rows = num_rows + 1;
5 }
```



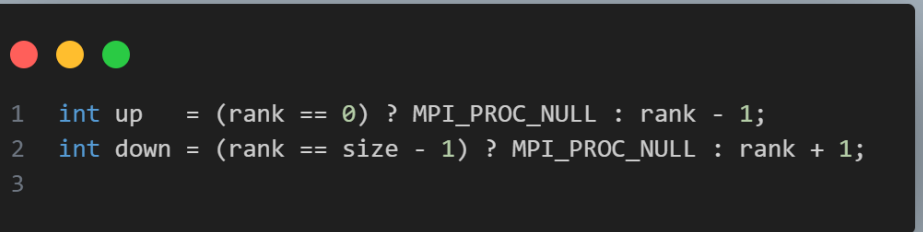
```
1 MPI_Scatterv(A, sendcounts, displs, MPI_INT,
2             &my_A[N], num_rows * N, MPI_INT,
3             0, MPI_COMM_WORLD);
```

DIVISIONE DEL LAVORO

- Quando ogni processo riceve la sua porzione di matrice A, per il calcolo delle corrispondenti righe di T sarà necessario gestire le righe «ghost» ovvero quelle necessarie per calcolare le righe di T, ma gestite da un altro nodo.



```
1 float* my_A = (float*) malloc((num_rows+2) * N * sizeof(float));
2 float* my_T = (float*) malloc(num_rows * N * sizeof(float));
```



```
1 int up   = (rank == 0) ? MPI_PROC_NULL : rank - 1;
2 int down = (rank == size - 1) ? MPI_PROC_NULL : rank + 1;
3
```

Per fare ciò, le righe corrispondenti della matrice A vengono salvate nella matrice my_A a partire dalla seconda riga (si guardi scatter). Dopodichè usando primitive di comunicazione punto a punto, ogni porcesso (tranne il primo e l'ultimo) ricevono la upper row e la bottom row dai processi adiacenti

DIVISIONE DEL LAVORO

- Primitive MPI_Sendrecv (operazione di comunicazione punto a punto combinata) tra i processi adiacenti, tranne ultima riga per ultimo processo e prima riga per il primo processo.
- La comunicazione è bidirezionale e sicura, evita deadlock tra i processi interagenti
- L'operazione è atomicamente definita e non è specificato l'ordine di esecuzione di send e receive. La semantica resta sempre **bloccante** e **buffer safe**.

```
1 // invio prima riga reale, ricevo ghost top
2 MPI_Sendrecv(&my_A[1 * N], N, MPI_INT, up, 0,
3             &my_A[0 * N], N, MPI_INT, up, 0,
4             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
5
6 // invio ultima riga reale, ricevo ghost bottom
7 MPI_Sendrecv(&my_A[num_rows * N], N, MPI_INT, down, 0,
8             &my_A[(num_rows + 1) * N], N, MPI_INT, down, 0,
9             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

```
1 //gather delle sottomatrici risultato
2 MPI_Gatherv(my_T, num_rows*N, MPI_INT, T, sendcounts, displs, MPI_INT, 0, MPI_COMM_WORLD);
```

- Per raccogliere i risultati parziali dei processi, nel processo main si usa MPI_Gatherv: gather con blocchi di dimensione variabile

ALGORITMO PARALLELO MPI

```
1  for (i = 1; i <= num_rows; i++) { //parto dalla seconda riga (altre ghost)
2      for (j = 0; j < N; j++) { //per ogni colonna
3
4          sum = 0.0;
5
6          for (ix = -radius; ix <= radius; ix++) {
7              for (iy = -radius; iy <= radius; iy++) {
8
9                  int y = i + iy; // righe (ghost-safe)
10                 int x = j + ix; // colonne
11
12                 x = mirror_col(x, N);
13                 y = clamp_row(y, num_rows, rank, size);
14
15                 sum += my_A[y * N + x];
16             }
17         }
18
19         med = sum / (ROUND * ROUND);
20         my_T[(i-1) * N + j] = my_A[i * N + j] > med;
21     }
22 }
```

- Gestione degli indici globali usando Mirror Index

```
1 // SOLO colonne
2 static inline int mirror_col(int x, int N) {
3     if (x < 0) return -x;
4     if (x >= N) return 2*N - x - 2;
5     return x;
6 }
```

```
1 // SOLO righe (clamp)
2 static inline int clamp_row(int y, int num_rows, int rank, int size) {
3     if (rank == 0 && y < 1) return 1;
4     if (rank == size - 1 && y > num_rows) return num_rows;
5     return y;
6 }
```

IMPLEMENTAZIONE OPEN MP

Libreria che permette di parallelizzare il codice di programmi C, utilizzando un approccio «dichiarativo», nel caso del modello a interazione con **memoria comune**. **Pertanto:**

- le matrici A e R siano condivise tra tutti i thread generati nel nodo con memoria comune
- ogni thread calcolerà gli elementi di R della porzione assegnatagli utilizzando i corrispondenti elementi della matrice dei dati A

DIVISIONE DEL LAVORO

La creazione dei thread paralleli è implicita e avviene per effetto della direttiva `#pragma omp parallel`, che con la clausola `num_threads` permette di specificare il grado di parallelismo.

Il numero di thread paralleli è p (compreso il master, valore parametrizzato). Il gruppo di thread paralleli è formato da 1 master e $(p-1)$ workers.

DIVISIONE DEL LAVORO

- Il codice dell'algoritmo è molto simile al caso MPI ma, in questo caso, A e T si trovano in memoria comune, pertanto la gestione risulta semplificata.



```
1 #pragma omp parallel num_threads(p) shared(A, R) private(i, j, my_rank) firstprivate(p, N, radius)
```

- Il workload viene parallelizzato con la clausola di `omp parallel for`



```
1 #pragma omp for schedule(static)
```

- Con la clausola `schedule` è stato possibile scegliere il tipo di scheduling.
 - `Static` prevede che la distribuzione del lavoro venga fatta assegnando i diversi chunks ai thread secondo una politica **round-robin**.
 - **N.B.:** se la dimensione del chunk non è specificata, OpenMP assegna alle thread chunk di dimensione approssimativamente uguale.
-

ALGORITMO

```
1  for (i = 0; i < N; i++) {
2      for (j = 0; j < N; j++) {
3
4          sum = 0.0;
5
6          for (ix = -radius; ix <= radius; ix++) {
7              for (iy = -radius; iy <= radius; iy++) {
8
9                  int row = i + ix;
10                 int col = j + iy;
11
12                 row = mirror_index(row, N);
13                 col = mirror_index(col, N);
14
15                 sum += A[row * N + col];
16             }
17         }
```

- Gestione indici globali: Mirror Index

```
1  int mirror_index(int idx, int N) {
2      if (idx < 0) return -idx;    // oob right
3      if (idx >= N) return 2*N - idx - 2; //oob left
4      return idx;
5  }
```

ANALISI DEI RISULTATI OTTENUTI

Misuriamo le prestazioni di entrambe attraverso l'analisi della scalabilità strong e weak di ognuna delle due soluzioni.

1. **Scalabilità strong:** è stato valutato quanto si può guadagnare nella sua soluzione mantenendo costante $N = 10000$ e aumentando il numero di processori. In MPI valutate le prestazioni su un unico nodo fisico, aumentando il numero di task per nodo (--ntasks-per-node=48 massimo valore per nodo).
2. **Scalabilità weak:** è stato valutato poi il guadagno, partendo da $N = 5000$ risolto su un nodo ed aumento linearmente la dimensione del problema al crescere del numero dei processori, mantenendo quindi pari a N il carico di ogni processore. Per farlo, è stato necessario calcolare $N = 5000 \cdot \sqrt{p}$

Scalabilità Strong:

- Speedup (S):
Def: $S(p, N) = \frac{T(1, N)}{T(p, N)}$
- Efficiency (E):
Def: $E = \frac{S}{p}$

Scalabilità Weak:

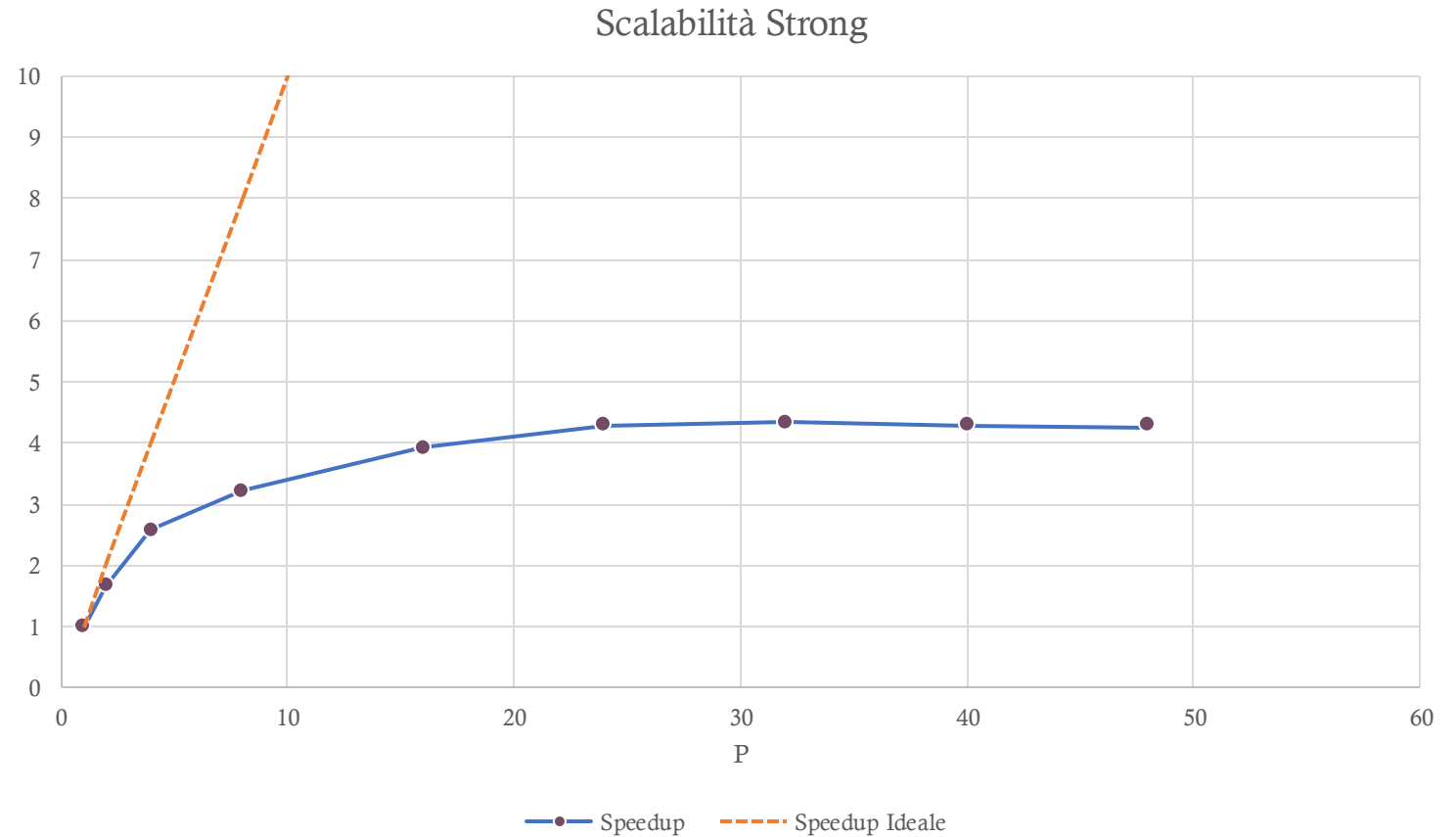
- Speedup (S)
Def: $S(p, N) = \frac{T(1, N)}{T(p, N)} \cdot p$
 - Efficiency (E):
Def: $E = \frac{S}{p}$
-

SCALABILITA' STRONG MPI

P	N	T(P,N)	Speedup	Efficiency
1	10000	1.473569	1	1
2	10000	0.881726	1.671232333	0.835616166
4	10000	0.570417	2.58331887	0.645829718
8	10000	0.456895	3.22518084	0.403147605
16	10000	0.375618	3.923052143	0.245190759
24	10000	0.343626	4.288293086	0.178678879
32	10000	0.339357	4.34223841	0.13569495
40	10000	0.343583	4.28829773	0.107220744
48	10000	0.343869	4.285262702	0.089276306

SCALABILITA' STRONG MPI

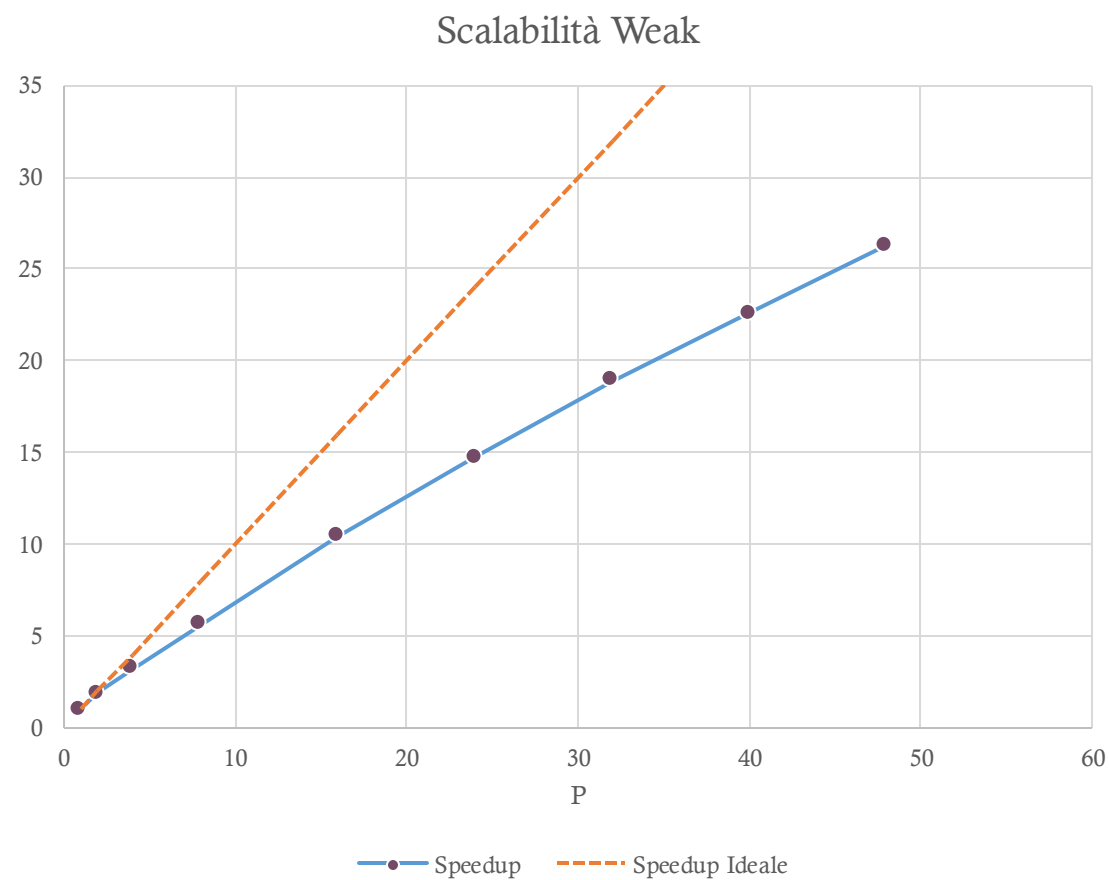
```
1  #!/bin/bash
2
3  #SBATCH --account=tra25_Infinfo
4  #SBATCH --partition=g100_usr_prod
5  #SBATCH --nodes=1
6  #SBATCH --ntasks-per-node=48
7  #SBATCH -o job.out
8  #SBATCH -e job.err
9  #SBATCH --mail-user=anna.ciampolini@unibo.it
10 module load autoload intelmpi
11
12 if [ $# -lt 1 ]; then
13     echo "Uso: sbatch launcher.sh <N>"
14     exit 1
15 fi
16
17 N=$1
18
19 echo "Strong Scalability Test"
20 echo "Matrix size: ${N}x${N}"
21
22 for p in 1 2 4 8 16 24 32 40 48; do
23     echo "Launching $p process on 1 node"
24     # srun con numero di processi e numero di nodi calcolato
25     srun -n $p ./bin $N
26 done
27
```



SCALABILITA' WEAK MPI

P	N	T(P,N)	Speedup	Efficiency
1	5000	1.390416	1	1
2	7071	1.509821	1.841828932	0.920914466
4	10000	1.717913	3.237453818	0.809363454
8	14142	1.9839	5.60679873	0.700849841
16	20000	2.1319	10.4351311	0.652195694
24	24494	2.269568	14.70323163	0.612634651
32	28284	2.352867	18.91025375	0.59094543
40	31622	2.465026	22.56229346	0.564057337
48	34641	2.540585	26.26952769	0.547281827

SCALABILITA' WEAK MPI

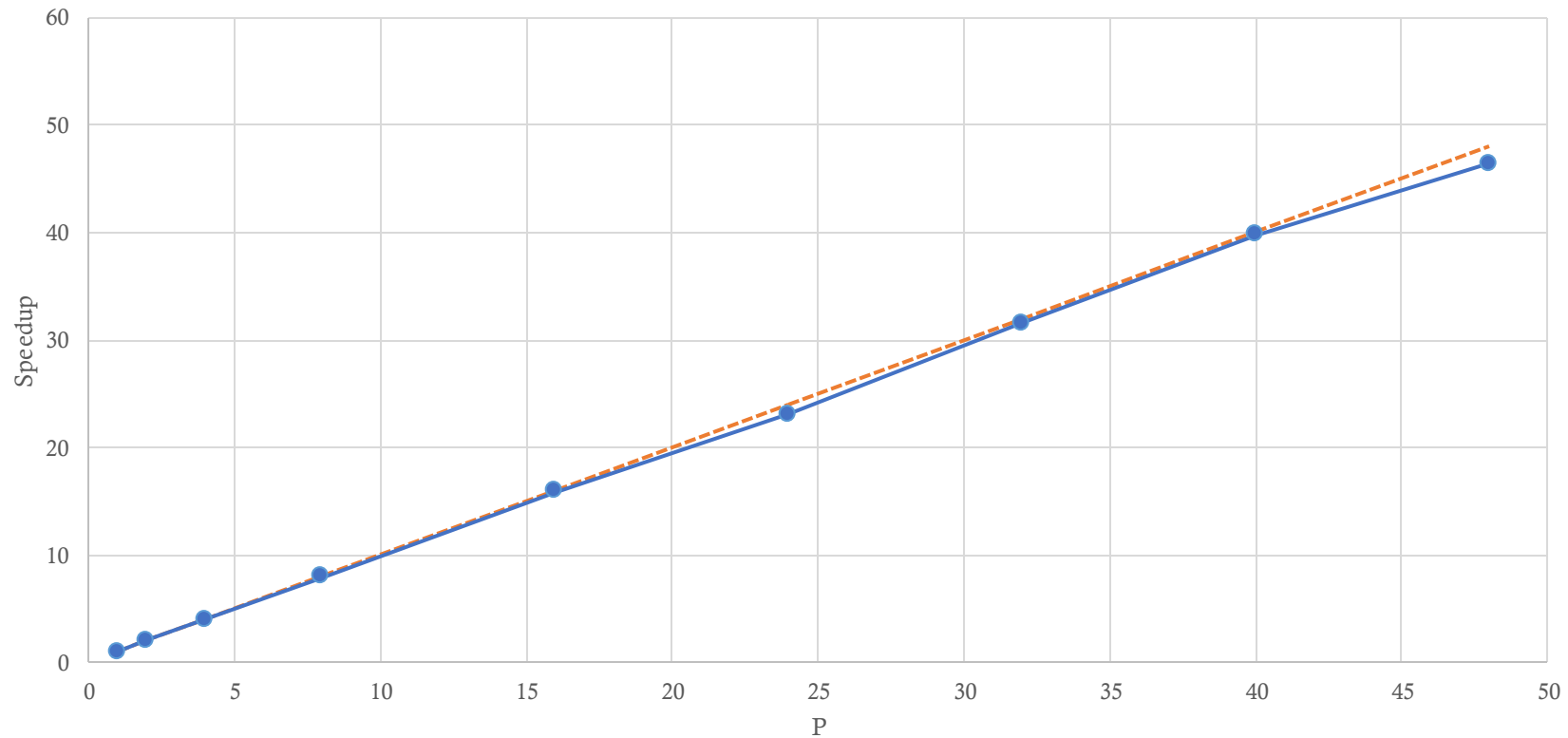


SCALABILITA' STRONG OPEN MP

P	N	T(P,N)	Speedup	Efficiency
1	10000	8.212742	1	1
2	10000	4.114754	1.995925394	0.997962697
4	10000	2.055701	3.995105319	0.99877633
8	10000	1.028794	7.982882871	0.997860359
16	10000	0.515017	15.94654545	0.996659091
24	10000	0.356194	23.05693527	0.960705636
32	10000	0.260184	31.56513083	0.986410338
40	10000	0.206696	39.73343461	0.993335865
48	10000	0.177078	46.37923401	0.966234042

SCALABILITA' STRONG OPEN MP

Scalabilità Strong



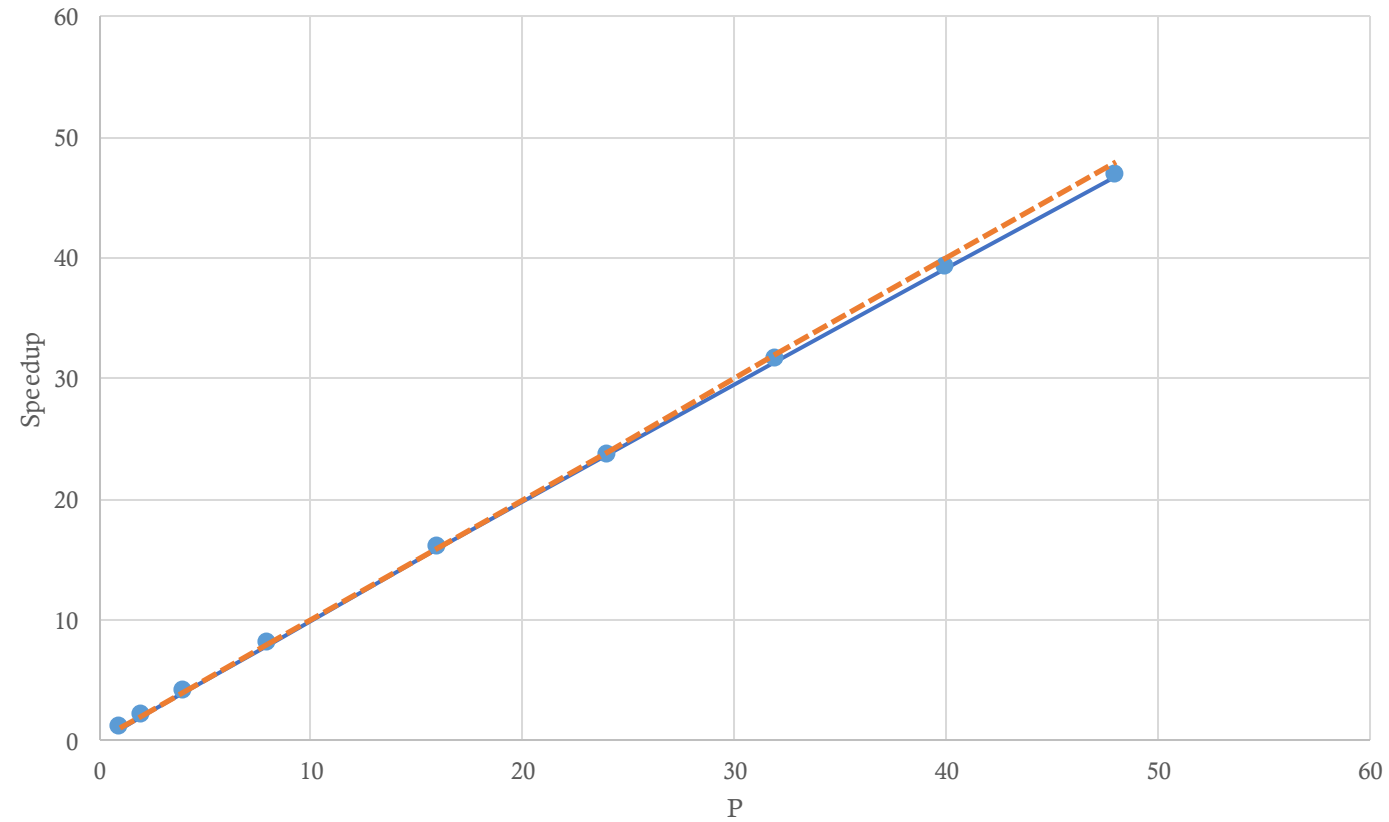
```
1 #!/bin/bash
2
3 #SBATCH --account=tra25 inginfo
4 #SBATCH --partition=g100_usr_prod
5
6 #SBATCH -t 00:05:00
7 #SBATCH --nodes=1
8 #SBATCH --ntasks-per-node=1 # Run a single task per node, more explicit than '-n 1'
9 #SBATCH -c 48 # number of CPU cores i.e. OpenMP threads per task
10 #SBATCH -o OMPjob.out
11 #SBATCH -e OMPjob.err
12
13 if [ $# -lt 1 ]; then
14     echo "Usage: sbatch launcher.sh <N>"
15     exit 1
16 fi
17
18 N=$1
19
20 echo "Strong Scalability Test"
21 echo "Matrix size: ${N}x${N}"
22
23
24 for I in 1 2 4 8 12 16 24 28 32 40 48; do
25     echo "Launching with $I OpenMP threads"
26     srun ./bin $I $N
27 done
28
```

SCALABILITA' WEAK OPEN MP

P	N	T(P,N)	Speedup	Efficiency
1	5000	2.056518	1	1
2	7071	2.057812	1.998742354	0.999371177
4	10000	2.056565	3.999908585	0.999977146
8	14142	2.056934	7.998382058	0.999797757
16	20000	2.059419	15.9774616	0.99859135
24	24494	2.085453	23.6670076	0.986125317
32	28284	2.089903	31.48881838	0.984025574
40	31622	2.102105	39.13254571	0.978313643
48	34641	2.11159	46.74812061	0.973919179

SCALABILITA' WEAK OPEN MP

Scalabilità weak



CONCLUSIONI

1. Programmazione parallela con **MPI**:

- Interazione basata su message passing su nodi distinti e bilanciamento a carico del programmatore determinano:
 - Speedup limitato ed efficienza decrescente in base al numero di nodi
 - Scalabilità e portabilità elevata, permette di eseguire su nodi multipli e anche su shared memory systems

2. Programmazione parallela con **OpenMP**:

- Interazione basata su memoria condivisa e load balancing semplice da realizzare determinano:
 - Speedup quasi ideali ed efficienza molto elevata
 - Scalabilità e portabilità limitata al numero di CPU core sul singolo nodo (massimo 48) ed esecuzione solo su shared memory systems.
-